

FrED MOBO — Algorithm and Software Architecture

Constrained two-objective Bayesian optimization of fiber extrusion with qLogNEHVI,
extensible to three objectives

Grupo Estudiantil FrED — Tecnológico de Monterrey, Campus Ciudad de México
Architecture record v1.4 — 18 September 2026 (adds target sessions and measured device constants)

Abstract

This document is the backbone specification for a locally-run Python application that executes a constrained multi-objective Bayesian optimization (MOBO) campaign on the FrED fiber-extrusion testbed. It fixes the mathematics, the data reduction, the constraint handling, the acquisition function (qLogNEHVI), the convergence rule, the export format for downstream AI agents, and the way the software modules interact. It is written to be read by three audiences at once: the student and the advisor, who must be able to audit every equation and every decision; and the coding agent (Claude Code), which will implement it. Every equation carries a source in Section B. Decisions that could reasonably have gone another way are flagged in boxes.

Contents

1	Scope, notation and standing decisions	2
1.1	What is optimized	2
1.2	Notation	2
1.3	Standing decisions inherited from the design record	3
2	Section A — Diameter data extraction	3
2.1	What the extractor receives and what it does not do	3
2.2	Extraction model	3
2.3	Window and operator declaration	4
2.4	Power objective: an independent logger, two sources by priority	5
3	Section B — Gaussian process surrogates	5
3.1	Model	5
3.2	Kernel (operator-selectable)	6
3.3	Hyperparameters by marginal likelihood (ML-II)	6
4	Section C — Constraints	7
4.1	Physics and triage (from the design record)	7
4.2	Mathematical form	7
4.3	What the constraint model must expose	8
5	Section D — MOBO with qLogNEHVI	8
5.1	Dominance, Pareto front, hypervolume	8
5.2	Acquisition function	9
5.3	The campaign loop	10
5.4	Convergence rule	10
5.5	Target sessions: one campaign, several target diameters	11
6	Section E — Persistence, export and the AI-agent format	11
6.1	Database, save and resume	11

6.2	Export format for downstream agents (Ollama / OpenRouter)	12
7	Section F — Algorithmic structure and module interaction	12
7.1	Modules (backend; the UI is a thin client over these)	12
7.2	Data flow	13
7.3	UI windows (specification only; implementation is Claude Code’s)	13
7.4	Extension to three objectives	13
8	BoTorch mapping (verified against the live documentation, 16 Sep 2026)	14
9	Implementation brief for Claude Code	14
9.1	Context	14
9.2	Non-negotiables	15
9.3	Repository layout (separate repo fred-mobo)	15
9.4	Milestones, in dependency order, each with its acceptance test	16
9.5	Pending hardware: what is built now and what will change	16
9.6	Session protocol (mandatory, because credits may end a session at any time)	17
9.7	Conventions	17
9.8	What not to do	17
A	Optional onset-suggestion detector (assist only)	17
B	Sources	18

1 Scope, notation and standing decisions

1.1 What is optimized

- **Design variables (phase 2D).** Heater temperature set-point T [°C] and spooler speed set-point ω_s [RPM]. Extrusion speed v_f (stepper) is held constant at $v_{f,0}$. Measured device constants: spool core diameter $D_{\text{spool}} = 15$ mm, extruded length per extruder revolution $\kappa_f = 50.27$ mm/rev. Nominal box: $T \in [70, 100]$, $\omega_s \in [25, 50]$. **Everything numeric in this document is a default that the operator can edit at campaign start:** bounds, d^* , $v_{f,0}$, $d_{\min}/d_{\max}$, reference point, adaptive budget. Only n_0 (initial Sobol runs) is fixed.
- **Objectives (phase 2D), both minimized.** f_1 = steady-state diameter error against the target d^* [mm]; f_2 = energy consumption, expressed as the mean total actuator power P [W] over the steady-state window (heater + spooler + extruder), i.e. energy per unit time, as defined in the design record.
- **Phase 3D (extension).** Add v_f as a third design variable and throughput $\dot{\ell}$ = spooled length per unit time (maximized, or equivalently $-\dot{\ell}$ minimized) as a third objective. The code must reach 3D by changing configuration, not code paths (Section 7.4).
- **Constraints.** Two failure modes bracket the draw ratio $R = v_s/v_f$: breakage (R too high) and overflow (R too low). They are handled as a learned outcome constraint (Section 4).

1.2 Notation

$\mathbf{x} \in \mathbb{R}^d$ is a design point in the *normalized* unit cube ($d = 2$ now, 3 later); $\mathbf{y} \in \mathbb{R}^m$ the vector of modeled outcomes (m = number of GP outputs); $\mathbf{f}(\mathbf{x}) \in \mathbb{R}^M$ the objective vector ($M = 2$ now); $\mathbf{r} \in \mathbb{R}^M$ the hypervolume reference point; $\mathcal{D}_n = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$ the data; μ_n, σ_n^2 the GP posterior mean and variance. BoTorch *maximizes*, so internally every minimized objective is negated; this document writes the mathematics in the “minimize” form the paper will use and states the sign flip once, in Section 8.

1.3 Standing decisions inherited from the design record

These were reached in earlier sessions and are not reopened here: (i) reference point fixed before the campaign, never recomputed from data; (ii) recommend the Pareto set from the posterior mean, never from raw observed y ; (iii) qLogNEHVI is the acquisition function, qNParEGO only as an offline baseline; (iv) objectives normalized before hypervolume; (v) one continuous constraint margin, not a binary label; (vi) feasibility inside the Monte-Carlo expectation; (vii) objective data from failed runs is logged but excluded from the objective GPs; (viii) experiments are sequential, $q = 1$; (ix) the temperature loop must be retuned before the campaign (prerequisite, listed in the experiments document, not part of this software); (x) failed runs do not count against the adaptive budget.

2 Section A — Diameter data extraction

2.1 What the extractor receives and what it does not do

Each run arrives as **two files**: the device CSV (semicolon-separated, decimal comma, columns as in `fred_experiment.test.csv`) and, when the power-measurement unit exists, the unit’s own log (Section 2.4; it runs on a separate microcontroller with its own clock). **The file contains steady-state operation only**: the operator starts logging after the diameter has settled, so the extractor has no transient to strip and no settling time to detect. Two properties of the file still drive the design, both measured on the test file:

1. **Sample-and-hold diameter.** The vision system updates at ≈ 3 Hz while the logger writes at 45 Hz: only 334 of 7814 diameter rows are new readings. Treating every row as an observation would overstate the sample size by $15\times$.
2. **Autocorrelation.** Distinct readings have lag-1 autocorrelation 0.89, so the standard error of the mean must use an effective sample size.

What the extractor deliberately does not do. It does not detect breakage or overflow, and it does not correct, mask or model temperature-loop disturbances. Temperature offsets are a tuning defect fixed upstream (experiments document, item E1); failures are *declared by the operator* (mode and approximate time) at intake. The extractor’s only job is a representative steady-state diameter and its uncertainty, under the assumption that the fiber neither broke nor overflowed inside the window it is given.

2.2 Extraction model

Let $\{(t_k, d_k)\}_{k=1}^N$ be the *distinct* raw-diameter readings inside the window $W = [t_a, t_b]$ (Algorithm 1). The per-run statistics are

$$\bar{d} = \frac{1}{N} \sum_{k=1}^N d_k, \quad s^2 = \frac{1}{N-1} \sum_{k=1}^N (d_k - \bar{d})^2, \quad (1)$$

$$b = \bar{d} - d^* \text{ (bias),} \quad \text{MAE} = \frac{1}{N} \sum_k |d_k - d^*|, \quad \text{RMSE} = \sqrt{b^2 + \frac{N-1}{N} s^2}, \quad (2)$$

$$\hat{\rho}_1 = \frac{\sum_{k=1}^{N-1} (d_k - \bar{d})(d_{k+1} - \bar{d})}{\sum_{k=1}^N (d_k - \bar{d})^2}, \quad N_{\text{eff}} = N \frac{1 - \hat{\rho}_1}{1 + \hat{\rho}_1}, \quad \text{SE}(\bar{d}) = \frac{s}{\sqrt{N_{\text{eff}}}}. \quad (3)$$

Equation (3) is the AR(1) effective-sample-size correction (Box–Jenkins). Without it the standard error is understated by a factor of ≈ 4 on the test file.

What the diameter GP models and what the objective is

The GP models the **smooth physical quantity** $\bar{d}(\mathbf{x})$, the steady-state mean diameter. Objective 1 is computed downstream as the absolute bias

$$f_1(\mathbf{x}) = |\bar{d}(\mathbf{x}) - d^*|,$$

evaluated *inside* the Monte-Carlo objective on posterior samples of $\bar{d}$ (Section 5.2). Reasons: (a) $|\cdot|$ has a crease at d^* that a stationary kernel cannot represent; a GP fitted directly to $|\bar{d} - d^*|$ fakes it with a short length-scale. (b) The same $\bar{d}$ GP carries the diameter-band feasibility constraint of Section 4. MAE and RMSE from (2) are stored and displayed but are not the modeled quantity. d^* is a campaign parameter.

2.3 Window and operator declaration

Algorithm 1: `extract_run(csv, declaration, cfg) → RunRecord`

Input: device CSV path; optional power-log path and alignment offset δ (Section 2.4); operator declaration $\{\text{status} \in \{\text{OK}, \text{BREAKAGE}, \text{OVERFLOW}\}, t_{\text{fail}}$ (confirmed or moved from the suggested onset), optional trim of both ends $[t_a, t_b]\}$; config: d^* , guard Δ_g , minimum window $W_{\min}$, plausibility $[d_{\text{lo}}^{\text{phys}}, d_{\text{hi}}^{\text{phys}}]$

Output: RunRecord

- 1 Parse with `sep=';', decimal=','`; validate required columns; map optional power columns (Section 2.4); `status ← SCHEMA_ERROR` and **return** on failure;
 - 2 Keep rows where `Diameter raw` changes value (distinct readings); mask readings outside the plausibility range (physically impossible values only, e.g. 0 mm from an occluded frame);
 - 3 $[t_a, t_b] \leftarrow$ operator trim if given, else $[t_{\text{start}}, t_{\text{end}}]$; if `status` $\neq$ OK: $t_b \leftarrow \min(t_b, t_{\text{fail}} - \Delta_g)$;
 - 4 **Onset suggestion (always shown):** run the detector of Appendix A; for a declared failure its result pre-fills t_{fail} because the operator will usually only know “it broke near the end”; the operator confirms or drags the marker. The confirmed value is what is stored;
 - 5 **if** $t_b - t_a < W_{\min}$ **then**
 - 6 `status ← TOO_SHORT`; **return**
 - 7 Compute (1)–(3) on readings in $[t_a, t_b]$;
 - 8 Diagnostics (logged and displayed, used by no model): $\bar{T}$, $\bar{\omega}_s$, their spreads, and a linear-trend check on d (warn if $|\text{slope}| \cdot (t_b - t_a) > 2 \text{SE}(\bar{d})$);
 - 9 Power: $\bar{P}$ over $[t_a + \delta, t_b + \delta]$ of the aligned power log (mode PM) or from the proxy (mode PX), per Section 2.4; record `power_source`;
 - 10 Draw ratio $R = \pi D_{\text{spool}} \omega_s / (\kappa_f \omega_f)$ from set-points ($D_{\text{spool}} = 15$ mm and $\kappa_f = 50.27$ mm/rev, measured with a vernier; at $\omega_f = 0.30$ RPM, $v_f = \kappa_f \omega_f = 15.1$ mm/min), stored on every run: it is the leading quantity for the constraint (Section 4);
 - 11 Constraint margin (mode A only): mean and peak spooler current in the window; for a declared failure, the values in the last Δ_g s before t_{fail} ;
 - 12 **return** RunRecord (status as declared; statistics of failed runs kept, flagged `partial`, excluded from objective GPs);
-

Defaults: $\Delta_g = 5$ s, $W_{\min} = 30$ s, plausibility $[0.05, 2.0]$ mm. On the test file, declaring BREAKAGE at $t_{\text{fail}} \approx 148$ s gives window $[0, 143]$ s, $\bar{d} \approx 0.43$ mm, $b \approx +0.08$ mm, $\hat{\rho}_1 \approx 0.9$, $N_{\text{eff}} \approx 10$; the trend check fires ($0.44 \rightarrow 0.40$ mm over the window). This is the acceptance test for `extraction.py`.

GP input is the set-point; achieved values are diagnostics only

The temperature-loop retune is a prerequisite that will be done before the campaign (experiments document, item M1), so the set-points $(T_{\text{sp}}, \omega_{s,\text{sp}})$ are the model inputs and the acquisition proposes in the same space: no mismatch to manage. Achieved means are stored and shown at intake with a warning if $|\bar{T} - T_{\text{sp}}| > \epsilon_T$, and the app only *suggests* rejecting such a run (operational decisions belong to the operator). The extractor never corrects, masks or models the temperature signal.

2.4 Power objective: an independent logger, two sources by priority

The power-measurement unit is a **separate microcontroller with its own clock and its own log**; it is not a plug-in of the control computer and is not synchronized with the device log. Whether it exists at campaign start is unknown, so both sources below are built and the one used is recorded per run.

Synchronization is not required. Both objectives are window means over tens of seconds, and the mode-A margin is also a window mean, so alignment to within about one second is sufficient. Two facts make even that cheap: the run window is a plateau, and the operator starts and stops both loggers by hand. The power log’s time base is elapsed seconds from its own start. The intake aligns the two files by an offset δ (power-log time minus device-log time) obtained, in order of preference, from (i) a run marker in both logs (see the PCB request below), (ii) the file creation timestamps, or (iii) the operator sliding the overlaid traces on the intake plot. The offset is stored with the run. Per-sample synchronization of current with RPM would only matter for a dynamic model of the motor, which nothing in this architecture uses.

Request to the PCB team (one-time, minimal). (a) Log a run identifier or a start/stop marker (a button press) as a row in the power log, so δ is read rather than estimated. (b) Log the supply voltage of each channel next to its current, since the voltage varies with operating point; power is then $V_c(t) I_c(t)$ per channel and needs no assumed voltage. (c) Confirm the stepper channel senses the supply side, not the coil side (design record open item 4). None of these requires synchronization with the device.

1. **Mode PM (preferred once available).** Power-log columns: a time column, and per channel **Heater current (A)**, **Extruder current (A)**, **Spooler current (A)** with their voltages **Heater voltage (V)** etc. (names are a config mapping, so the unit’s actual headers are accepted without code change). $P(t) = \sum_c V_c(t) I_c(t)$; $\bar{P}$ = mean over the aligned window. Heater current averages over whole 1 Hz PWM periods automatically since the window $\gg 1$ s.
2. **Mode PX (proxy, until the unit exists).** Heater: $P_h(t) = \frac{u(t)}{100} P_{h,\text{max}}$ from the logged, clamped PID output, with $P_{h,\text{max}} = V_h^2 / R_h$ measured once (experiments M3). Spooler: from the calibration table current-vs-RPM with fiber, taken during the same bench session (M3), times the measured supply voltage. Extruder: constant $P_{f,0}$ in phase 2D, from the same session.

The database column `power_source` $\in \{\text{PM}, \text{PX}\}$ prevents silent pooling. Mode PX cannot see load-dependent heater power; it is a bridge, and the paper should report which runs used it.

3 Section B — Gaussian process surrogates

3.1 Model

Each modeled outcome $y^{(j)}$, $j = 1..m$, gets an independent GP (a `ModelListGP`); $m = 3$ in mode A (diameter, power, margin) and $m = 2$ in mode B. With inputs normalized to $[0, 1]^d$ and outcomes standardized (BoTorch default), the prior is $f \sim \mathcal{GP}(0, k)$ and the observation model $y = f(\mathbf{x}) + \varepsilon$,

$\varepsilon \sim \mathcal{N}(0, \sigma_n^2)$. Given data $X, \mathbf{y}$ and test point $\mathbf{x}_*$ the posterior is Gaussian with (R&W eqs. 2.23–2.24)

$$\mu_n(\mathbf{x}_*) = \mathbf{k}_*^\top (K + \sigma_n^2 I)^{-1} \mathbf{y}, \quad (4)$$

$$\sigma_n^2(\mathbf{x}_*) = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^\top (K + \sigma_n^2 I)^{-1} \mathbf{k}_*, \quad (5)$$

where $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ and $[\mathbf{k}_*]_i = k(\mathbf{x}_i, \mathbf{x}_*)$.

3.2 Kernel (operator-selectable)

Two stationary ARD kernels are offered in the settings; the choice is a campaign parameter, stored with every fit, and the kernel window renders whichever is active. Default: Matérn-5/2 with automatic relevance determination and an output scale (R&W eq. 4.17 for the $\nu = 5/2$ form; ARD as in R&W §5.1):

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \left(1 + \sqrt{5} r + \frac{5}{3} r^2 \right) e^{-\sqrt{5} r}, \quad r^2 = \sum_{i=1}^d \frac{(x_i - x'_i)^2}{\ell_i^2}. \quad (6)$$

Alternative: the squared-exponential (SE, “RBF”) kernel (R&W eq. 4.9 with ARD, §5.1):

$$k_{\text{SE}}(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp \left(-\frac{1}{2} \sum_{i=1}^d \frac{(x_i - x'_i)^2}{\ell_i^2} \right). \quad (7)$$

Hyperparameters in both cases: $\theta = (\sigma_f, \ell_1, \dots, \ell_d, \sigma_n)$. Matérn-5/2 is the default because it assumes only twice-differentiable sample paths, a weaker smoothness claim than the infinitely-smooth SE and easier to defend near a failure boundary; the SE option lets the operator test that claim (a visibly worse leave-one-out residual under SE is a reportable result). The kernel is chosen at campaign setup and **locked once the first adaptive run is accepted**; the simulation tab remains free to compare both. Before the lock, a switch refits from scratch and is logged.

Explicit kernel, inferred homoscedastic noise, per-run SE stored but not injected

BoTorch’s current `SingleTaskGP` default kernel and priors follow Hvarfner et al. (2024). We override the kernel with (6) explicitly so that what the paper states is what runs. Noise: the within-run standard error (3) captures measurement scatter only; the dominant noise on this device is run-to-run (the design record documents replicate spread exceeding the across-condition spread). We therefore let σ_n^2 be inferred by ML-II (default), and keep `train_Yvar` = SE^2 as a switchable option for a sensitivity check.

3.3 Hyperparameters by marginal likelihood (ML-II)

The log marginal likelihood (R&W eq. 2.30) and its gradient (R&W eqs. 5.8–5.9) are

$$\log p(\mathbf{y} \mid X, \theta) = -\frac{1}{2} \mathbf{y}^\top K_y^{-1} \mathbf{y} - \frac{1}{2} \log |K_y| - \frac{n}{2} \log 2\pi, \quad K_y = K_\theta + \sigma_n^2 I, \quad (8)$$

$$\frac{\partial}{\partial \theta_j} \log p(\mathbf{y} \mid X, \theta) = \frac{1}{2} \text{tr} \left[(\boldsymbol{\alpha} \boldsymbol{\alpha}^\top - K_y^{-1}) \frac{\partial K_y}{\partial \theta_j} \right], \quad \boldsymbol{\alpha} = K_y^{-1} \mathbf{y}. \quad (9)$$

`fit_gpytorch.mll` maximizes (8) (plus the log priors) with L-BFGS-B. The three terms of (8) are what the “kernel window” of the app displays: data fit, complexity penalty, and their sum, on a 2-D grid over (ℓ_1, ℓ_2) at the fitted σ_f, σ_n , with the optimum marked. This makes “how the kernel parameters were reached” visible and auditable.

Algorithm 2: `fit_models(D) → ModelListGP, fit_report`

```
1  $X \leftarrow \text{normalize}(\text{set-point inputs, bounds})$ ; for each outcome  $j$ :  $Y_j \leftarrow \text{column } j$  (objective
   GPs use OK runs only; margin GP uses OK and failed runs);
2 for  $j = 1..m$  do
3   build SingleTaskGP( $X, Y_j, \text{covar\_module}=\text{ScaleKernel}(\text{Matern}(\nu=2.5,$ 
     ard_num_dims= $d$ ) or RBFKernel(ard_num_dims= $d$ ) per config),
     outcome_transform=Standardize);
4 model  $\leftarrow \text{ModelListGP}(\text{models})$ ; mll  $\leftarrow \text{SumMarginalLogLikelihood}$ ;
5 fit_gpytorch_mll(mll) with  $n_{\text{restarts}}$  random restarts; keep best (8);
6 fit_report  $\leftarrow \{\theta_j, \text{LML}_j, \text{LML grid over } (\ell_1, \ell_2), \text{leave-one-out residuals}\}$  per outcome;
7 return model, fit_report;
```

Models are re-initialized and refit from scratch after every new run (no warm start of hyperparameters), following the BoTorch tutorial finding that warm-starting hurts on small data.

4 Section C — Constraints

4.1 Physics and triage (from the design record)

Under steady-state mass conservation $d_{\text{out}} = d_{\text{in}}/\sqrt{R}$ with $R = v_s/v_f$; breakage occurs at high R in the still-molten neck (temperature-dependent), overflow at low R . In phase 2D, v_f is fixed, so $R \propto \omega_s$ and the two thresholds are, to first order, two values of ω_s whose position moves with T . Spooler stall and failure to reach steady state are not modeled (known constraint / protocol matter).

4.2 Mathematical form

Let $c(\mathbf{x})$ be a smooth, monotone-in- R margin. Feasibility is the band

$$\mathcal{F} = \{\mathbf{x} : \tau_{\text{lo}} \leq c(\mathbf{x}) \leq \tau_{\text{hi}}\}, \quad (10)$$

which is expressed to BoTorch as **two** outcome constraints on **one** GP output, each in the convention “ $g(\mathbf{y}) \leq 0$ is feasible”:

$$g_1(\mathbf{y}) = \tau_{\text{lo}} - y_c, \quad g_2(\mathbf{y}) = y_c - \tau_{\text{hi}}. \quad (11)$$

Feasibility enters the acquisition as $\mathbb{E}[\text{HVI} \cdot \mathbb{I}\{g_1 \leq 0\} \mathbb{I}\{g_2 \leq 0\}]$ with the indicators evaluated on each joint posterior sample (Section 5.2), never as $\mathbb{E}[\text{HVI}] \cdot \Pr(\mathcal{F})$: the design record shows the product form overstates by 57 % under strong negative correlation, always toward riskier settings.

Declared failures, draw ratio as the leading quantity, two modes selected by configuration

Failures are declared, not detected. At intake the operator states the mode (breakage / overflow) and the approximate time. The app records the run’s draw ratio R with that label. Because v_f is fixed in phase 2D, R is a known, exact function of the set-point ω_s ; the unknown is only *where* in R the two boundaries sit and how they move with T .

Mode B (default, available today): R -band from declared failures + diameter band on $\bar{d}$. (i) The app maintains a temperature-dependent band: at temperature T , $R_{hi}(T) = \min\{R_i : \text{breakage declared at } (R_i, T_i), |T_i - T| \leq \Delta T\}$ and $R_{lo}(T) = \max\{R_i : \text{overflow at } (R_i, T_i), |T_i - T| \leq \Delta T\}$, with $\Delta T = 5^\circ\text{C}$ by default (nearby temperatures are assumed to share similar bands; ΔT editable). After each declared failure the app *suggests* the corresponding ω_s limit for that temperature neighbourhood and never applies it. Accepted limits enter `optimize_acqf` as a piecewise-linear input constraint in (ω_s, T) (non-linear_inequality_constraints), or, when a single pair covers the whole T range, as a plain bound. An accepted suggestion becomes a known input constraint at zero modeling cost. (ii) The outcome constraint is the diameter band $d_{\min} \leq \bar{d}(\mathbf{x}) \leq d_{\max}$ on the already-modeled diameter, implemented as (11) with $y_c = \bar{d}$; defaults $0.6 d^*$ and $1.6 d^*$, editable. This encodes the physics ($d_{\text{out}} = d_{\text{in}}/\sqrt{R}$: too thin breaks, too thick falls) and carries the temperature dependence through the GP. Failed runs do not enter any GP in mode B.

Mode A (enabled by config when the spooler current channel exists): margin GP. y_c = mean spooler armature current in the window ($\propto$ torque $\propto$ tension $\propto$ breakage margin), thresholds τ_{lo}, τ_{hi} from the torque sweep (experiments document, session M3b), editable. Declared failures *do* contribute: the current in the last Δ_g s before the declared t_{fail} is the margin at failure, the most informative observation available. The R -band suggestion of mode B stays active in mode A.

Same code path in both modes: a config entry names the outcome column carrying the constraint and the two thresholds.

4.3 What the constraint model must expose

Probability of feasibility on the design-space grid, $\Pr(\tau_{lo} \leq c(\mathbf{x}) \leq \tau_{hi}) = \Phi\left(\frac{\tau_{hi}-\mu_c}{\sigma_c}\right) - \Phi\left(\frac{\tau_{lo}-\mu_c}{\sigma_c}\right)$, displayed as a heat-map with the declared failures (by R and T) and the current R -band overlaid, so that “how the algorithm avoids the failure regions” is visible.

5 Section D — MOBO with qLogNEHVI

5.1 Dominance, Pareto front, hypervolume

For minimization, $\mathbf{f}(\mathbf{x})$ *dominates* $\mathbf{f}(\mathbf{x}')$, written $\mathbf{f} \prec \mathbf{f}'$, iff $f_j \leq f'_j \forall j$ and $f_j < f'_j$ for some j . The Pareto set is the set of non-dominated points; its image is the Pareto front $\mathcal{P}$. The hypervolume indicator with reference point $\mathbf{r}$ (Zitzler & Thiele 1999) is

$$\text{HV}(\mathcal{P}; \mathbf{r}) = \lambda_M \left(\bigcup_{\mathbf{f} \in \mathcal{P}} \{\mathbf{f}' \in \mathbb{R}^M : \mathbf{f} \preceq \mathbf{f}' \preceq \mathbf{r}\} \right), \quad (12)$$

the Lebesgue measure of the region dominated by $\mathcal{P}$ and bounded by $\mathbf{r}$. The hypervolume improvement of a candidate set $\mathcal{Y}$ is $\text{HVI}(\mathcal{Y} \mid \mathcal{P}) = \text{HV}(\mathcal{P} \cup \mathcal{Y}) - \text{HV}(\mathcal{P})$ (Emmerich et al. 2006).

Normalization before HV. Objectives are mapped to $\tilde{f}_j = (f_j - f_j^{\text{ideal}})/(f_j^{\text{nadir}} - f_j^{\text{ideal}})$ with fixed, pre-campaign $f_j^{\text{ideal}}, f_j^{\text{nadir}}$ from pilot ranges, and $\mathbf{r}$ is specified in raw units and mapped by the same affine map. Without this a large constant power baseline makes the acquisition insensitive to real differences (design record §4).

Reference point

$\mathbf{r} = (0.15 \text{ mm}, P_{\text{ref}})$: the error side comes from the design record (worst observed run 0.130 mm); the power side is a placeholder until the first PM-mode pilot, after which it is frozen at “worst pilot value +10 %” and written into the campaign config with a timestamp. The UI allows editing but every edit is logged, and the app refuses to compute a hypervolume history across a reference-point change without re-computing the whole history under the new point.

5.2 Acquisition function

From EHVI to qNEHVI. With posterior samples $\mathbf{f}^{(s)} \sim p(\mathbf{f} \mid \mathcal{D}_n)$ drawn jointly at the n evaluated points *and* the q candidates, the noisy expected hypervolume improvement (Daulton et al. 2021, eq. 4 of that paper, written here for our notation) is

$$\alpha_{\text{qNEHVI}}(\mathbf{x}_{1:q}) = \mathbb{E}[\text{HVI}(\mathbf{f}(\mathbf{x}_{1:q}) \mid \mathcal{P}_n^{(s)})] \approx \frac{1}{S} \sum_{s=1}^S \text{HVI}(\mathbf{f}^{(s)}(\mathbf{x}_{1:q}) \mid \mathcal{P}_n^{(s)}), \quad (13)$$

where $\mathcal{P}_n^{(s)}$ is the Pareto front rebuilt from the *same* sample s at the already-evaluated inputs. Rebuilding the front per sample is what removes the winner’s-curse bias of plain EHVI under noise. With constraints the summand becomes $\text{HVI}(\cdot) \prod_k \mathbb{I}\{g_k(\mathbf{y}^{(s)}) \leq 0\}$, i.e. the feasibility of the candidate under the *same* joint sample. Samples use the reparameterization $\mathbf{f}^{(s)} = \mu + L \boldsymbol{\epsilon}^{(s)}$ with fixed quasi-Monte-Carlo base samples $\boldsymbol{\epsilon}^{(s)}$ (Sobol), so that (13) is a deterministic, differentiable function of $\mathbf{x}$ and can be maximized with L-BFGS-B (Daulton et al. 2020).

The Log reformulation. HVI in (13) is a sum over hypervolume cells of products of clamped differences $\max(0, \cdot)$; far from the front the value underflows to zero and the gradient vanishes. Ament et al. (2023) replace $\max(0, z)$ by a smooth softplus $\tau \log(1 + e^{z/\tau})$, the average over samples by a log-sum-exp, and products by sums of logs, giving $\log \alpha$ directly:

$$\log \alpha_{\text{qLogNEHVI}} = \text{logsumexp}_s \left[\log \text{HVI}_s^{\text{soft}} + \sum_k \log \sigma(-g_k(\mathbf{y}^{(s)})/\eta) \right] - \log S, \quad (14)$$

where $\sigma(\cdot)$ is the logistic function with temperature η used as a smooth feasibility indicator. This is an optimization device, not a better estimator: $\exp$ of (14) equals (13) only in the limit $\tau, \eta \rightarrow 0$. The paper should say so.

Objective transform and constraint callables as passed to BoTorch

Model outputs are $\mathbf{y} = (\bar{d}, P, c)$ (mode A) or $(\bar{d}, P)$ (mode B). The MC objective maps samples of $\mathbf{y}$ to the *maximized* objective vector $(-|\bar{d} - d^*|, -P)$; constraints are callables on $\mathbf{y}$ returning (11). The Pareto front, hypervolume and reference point therefore all live in $(|\bar{d} - d^*|, P)$ space, as the paper will report them.

5.3 The campaign loop

Algorithm 3: `campaign()` — outer loop (sequential, $q = 1$ by default)

Input: config: bounds, d^* , $\mathbf{r}$, normalization ranges, constraint mode and thresholds, n_0 , budget $N_{\max}$, q , MC samples S , stopping rule $(\epsilon_{\text{HV}}, k, \epsilon_\alpha)$, seed

- 1 **Initialization.** Draw n_0 Sobol points in the normalized box (`draw_sobol_samples`); enforce that the extreme ω_s values are present (see remark); present them as the first experiment list;
- 2 **while** $n < N_{\max}$ *and not converged* **do**
- 3 Operator runs the experiment, uploads the CSV; `extract_run` $\rightarrow$ RunRecord; store in DB;
- 4 **if** `status` $\neq$ OK **then**
- 5 log; in mode A append $(\mathbf{x}, c)$ to constraint data; else skip; continue only if the operator confirms;
- 6 model, report $\leftarrow$ `fit_models`($\mathcal{D}_n$);
- 7 `acq` $\leftarrow$ `qLogNEHVI`(model, $\mathbf{r}$, X_{baseline} =all OK inputs, objective, constraints, `prune.baseline`=True, `sampler`=Sobol(S));
- 8 $\mathbf{x}_{\text{next}}$, $\log \alpha^*$ $\leftarrow$ `optimize_acqf`(`acq`, unit-cube bounds, q , `num_restarts`, `raw_samples`, `sequential`=True);
- 9 Unnormalize $\mathbf{x}_{\text{next}}$ to set-points; compute displays (Section 7.3): acquisition surface, posterior means/stds, $\text{Pr}(\mathcal{F})$, current fronts;
- 10 $\text{HV}_n \leftarrow$ hypervolume of the observed feasible front; $\widehat{\text{HV}}_n \leftarrow$ hypervolume of the posterior-mean front on a grid; `converged` $\leftarrow$ rule of Section 5.4;
- 11 Persist everything (Section 6);
- 12 Final: Pareto set from posterior mean (never from raw y), with per-point posterior std; export (Section 6.2);

Remark on initialization and budget. $n_0 = 2(d+1) = 6$ for $d = 2$ (BoTorch tutorial convention) is fixed. The adaptive budget defaults to 20 additional runs and is editable; **declared failures do not count** (they usually fail early and cost little), so the loop counts OK runs only. The design record (open item 8) warns that with only two objectives nothing rewards high speed, so the constraint boundary at high ω_s would be under-sampled; forcing the two extreme ω_s values into the initial design at mid-temperature is the cheapest fix and is adopted.

Remark on q . q is exposed in the UI but changing it from 1 triggers a confirmation dialog stating that (i) the device runs one experiment at a time, (ii) a batch is only useful if all q runs will be executed before the next fit, and (iii) with $q > 1$ the candidates are produced by sequential greedy optimization (`sequential`=True) and shown as an ordered list.

5.4 Convergence rule

Three signals are computed every iteration; the app declares **converged** when (a) and (b) hold for k consecutive iterations, or (c) holds:

$$(a) \quad \frac{\text{HV}_n - \text{HV}_{n-k}}{\text{HV}_n} < \epsilon_{\text{HV}} \quad (\text{observed front has stopped growing}), \quad (15)$$

$$(b) \quad \exp(\log \alpha_n^*) < \epsilon_\alpha \text{HV}_n \quad (\text{the best candidate cannot add more than a fraction of the current HV}), \quad (16)$$

$$(c) \quad n \geq N_{\max}. \quad (17)$$

Defaults $\epsilon_{\text{HV}} = 0.02$, $\epsilon_\alpha = 0.01$, $k = 3$. Signal (b) is the natural stopping statistic for an improvement-based acquisition: it is the expected gain of the next experiment in the same units as the front. Both signals are plotted against n in the convergence window; when (a)–(b) trigger the app prompts the operator to export.

5.5 Target sessions: one campaign, several target diameters

The GPs never see d^* : they model $\bar{d}(\mathbf{x})$ and $\bar{P}(\mathbf{x})$, and the target enters only in the objective transform $f_1 = |\bar{d} - d^*|$ and in the diameter band $[0.6 d^*, 1.6 d^*]$. A campaign is therefore organized as a sequence of **target sessions** that share one dataset, one set of GPs, one kernel and one reference point:

1. **Session 1** at d_1^* runs Algorithm 3 as written (initial design + adaptive runs until the convergence rule fires). Its by-products are the maps $\mu_d, \sigma_d, \mu_P, \sigma_P$ over the whole box, tight near the d_1^* band and loose elsewhere.
2. **Predicted front for a new target.** For a candidate d_2^* inside $[d_{\min}, d_{\max}]$ the app inverts the diameter map: for each T on a grid it solves $\mu_d(T, \omega_s) = d_2^*$ for ω_s (bracketing root search; initial guess from mass conservation, $\omega_s \propto d^{-2}$), giving the *iso-diameter curve* $\omega_s^{\text{iso}}(T; d_2^*)$. Reading $\mu_P, \sigma_d, \sigma_P$ and $\text{Pr}(\mathcal{F})$ along that curve yields a predicted front with an uncertainty band, before any experiment. Wide σ means the device has not been operated there.
3. **Session 2** is the same loop continued with the transform switched to $|\bar{d} - d_2^*|$: no run is discarded, nothing is refit for the switch, hyperparameters stay locked. Because σ_d is large along the new curve, the first proposals land on it and are corrected by what comes back; convergence is tested per session with the same rule, and sessions after the first typically need far fewer runs. The R -band from declared failures carries over (it is physics, not target); the diameter-band constraint rescales automatically.
4. **Coverage.** Three sessions (low, middle, high d^* in the allowed band) leave the maps well-sampled everywhere the downstream agent may be asked to operate; further sessions refine.

Hypervolume comparability. The error axis of $\mathbf{r}$ (0.15 mm) and the power axis are the same for every session, so HV_n curves are comparable across sessions; each session keeps its own history and its own frozen front.

What the downstream agent does with this, and what it does not

The agent (LLM tuner, outside this software, active only after the campaign) improves FrED’s operation using the exported **maps**, not the fronts: for any requested d^* it computes the iso-diameter curve and predicted front itself, chooses an operating point on it according to its preference (accuracy vs. energy), and when the observed steady-state diameter differs from the target it corrects ω_s with the map’s local slope $\partial\mu_d/\partial\omega_s$ (analytically $\approx -\bar{d}/(2\omega_s)$), bounded by the R -band and the diameter band so it can never step into a failure region. This is a slow supervisory loop on set-points above the fast PIDs. The agent never proposes experiments, never modifies fronts or GPs, and interpolation between sessions is done on the maps, because a curve “between two fronts” has no meaning in objective space.

6 Section E — Persistence, export and the AI-agent format

6.1 Database, save and resume

A campaign takes several days. The campaign file is therefore the single source of truth and the app must close and reopen at any point without loss: **Open campaign** restores config, every run, every fit report, the reference-point / bounds / kernel history, the current proposal and the convergence state; models are refit deterministically from the stored data and seed (nothing is pickled), and a session log records each open and close. Autosave after every intake and every fit. Raw CSVs are copied into the campaign folder so the file is self-contained and portable between machines.

SQLite file per campaign, with tables **campaign** (config JSON, reference point history, normalization ranges, seed), **runs** (one row per RunRecord: set-points, achieved inputs, $\bar{d}$, s , N , N_{eff} , bias, MAE, RMSE, $\bar{P}$, power_source, margin, status, partial flag, raw-CSV path and hash), **sessions** (one row per target session: d^* , start/end run ids, convergence state, frozen front id), **fits** (per iteration:

session id, hyperparameters per outcome, LML, HV_n , $\widehat{HV}_n$, $\log \alpha^*$, proposed candidate(s)). Raw CSVs are copied into the campaign folder; nothing is recomputed from memory.

6.2 Export format for downstream agents (Ollama / OpenRouter)

The LLM tuner in the controller architecture consumes “Pareto point targets per objective” and needs the mathematical context of each point. Three files are exported together; the JSON is the machine-readable source of truth and the Markdown is its human/LLM-readable rendering (LLMs reason better over prose-plus-tables than over raw JSON, and JSON is what a tool-calling agent can parse deterministically):

- **campaign_export.json** — schema versioned; keys: **meta** (device, date, software versions, seed), **config** (bounds, d^* , **r**, normalization, constraint mode/thresholds), **runs[]** (every RunRecord), **model** (kernel form, hyperparameters per outcome, LML), **sessions[]** (one entry per target session: d^* , **pareto_posterior[]** with set-points, posterior mean and std of every outcome, $\Pr(\mathcal{F})$ and objective values in raw and normalized units; **pareto_observed[]**; **history[]** with HV_n , $\widehat{HV}_n$, $\log \alpha_n^*$), **maps** (mandatory: $\mu_d, \sigma_d, \mu_P, \sigma_P$, $\Pr(\mathcal{F})$ and the R -band on a $G \times G$ grid over (T, ω_s) ; this is what the agent computes with, Section 5.5), **iso_diameter[]** (for each session target and for a user-chosen list of extra d^* values: the predicted iso-diameter curve and predicted front with uncertainty).
- **campaign_summary.md** — the same content as prose and Markdown tables, plus the equations of this document in one paragraph each, so an agent can explain a recommendation.
- **pareto_points.csv** — flat table for spreadsheets.

Every exported point carries its uncertainty; an agent that picks a set-point from the front is told how sure the model is.

7 Section F — Algorithmic structure and module interaction

7.1 Modules (backend; the UI is a thin client over these)

Module	Responsibility	Public interface
<code>config.py</code>	Typed campaign configuration; validation; JSON round-trip; dimension-agnostic (lists of variables and objectives)	<code>CampaignConfig</code> , <code>load/save</code>
<code>extraction.py</code>	Section A	<code>extract_run(csv, cfg) -> RunRecord</code>
<code>space.py</code>	Variable definitions, coordinate map (raw $\leftrightarrow$ (log ω_s , T) $\leftrightarrow$ unit cube), input constraints for 3D	<code>to_unit</code> , <code>from_unit</code> , <code>bounds</code> , <code>linear_constraints</code>
<code>models.py</code>	Section B	<code>fit_models(data, cfg) -> (model, FitReport)</code>
<code>constraints.py</code>	Section C: builds constraint callables from config; feasibility probability	<code>make_constraints(cfg)</code> , <code>prob_feasible(model, X)</code>
<code>acquisition.py</code>	Section D: objective transform, qLogNEHVI construction, optimization	<code>propose(model, data, cfg) -> Proposal</code>
<code>pareto.py</code>	Non-dominance, hypervolume, posterior-mean front on grid, convergence signals	<code>observed_front</code> , <code>posterior_front</code> , <code>hv</code> , <code>convergence</code>
<code>storage.py</code>	Section E database	<code>Campaign</code> , <code>add_run</code> , <code>add_fit</code> , <code>export</code>
<code>campaign.py</code>	Algorithm 3 orchestration; the only module the UI calls	<code>CampaignController</code>
<code>simulate.py</code>	Synthetic FrED emulator for tests and the qNParEGO baseline (of-fine only)	<code>FakeFrED</code>

7.2 Data flow

```

device CSV + power log (+ offset) --extraction.py--> RunRecord --storage.py--> runs table
runs table --space.py (achieved -> unit cube)--> (X, Y, status)
(X, Y) --models.py--> ModelListGP + FitReport -----+
FitReport --> [kernel window] |
model --constraints.py--> Pr(feasible) grid --> [design-space window]
model + config --acquisition.py--> Proposal(x_next, log_alpha*) --> [next-experiment
    window]
model + runs --pareto.py--> HV_n, HVhat_n, fronts --> [Pareto window], [convergence
    window]
all of the above --storage.py--> fits table, export (JSON/MD/CSV) --> AI agent

```

7.3 UI windows (specification only; implementation is Claude Code’s)

1. **Campaign / settings** (English UI): new / open / save-as campaign; **target sessions**: list of sessions with their d^* and state, “predict front for new d^* ” and “start session at d^* ” actions; variable bounds, $v_{f,0}$, D_{spool} , d^* (all editable at start; later edits are logged and refit), reference point (power side entered by the operator), normalization ranges, constraint mode and thresholds ($d_{\min}, d_{\max}, \tau_{\text{lo}}, \tau_{\text{hi}}$), kernel choice, q (with the confirmation dialog), adaptive budget, stopping parameters, seed.
2. **Run intake**: device-CSV picker and optional power-log picker with alignment control (offset δ from run markers or set by hand on an overlaid plot); declaration form (status; t_{fail} pre-filled by the onset suggestion, draggable; trim of both ends); extraction preview (diameter trace with window and markers, aligned power trace, achieved T/ω_s as diagnostics, statistics); accept/reject; after a declared failure, the R -band suggestion dialog.
3. **Pareto window**: observed feasible front and posterior-mean front in $(|\bar{d} - d^*|, P)$, colored by iteration, reference point drawn, dominated region shaded; slider to replay the front by iteration.
4. **Design-space window**: 2-D heat-maps over (ω_s, T) of $\log \alpha$, posterior mean and std of each outcome, $\Pr(\mathcal{F})$; evaluated points, failures and the next candidate overlaid. In 3D: slices at a chosen third coordinate.
5. **GP window** (separate window): per outcome, posterior mean surface with $\pm 2\sigma$ band along a user-chosen 1-D cut, and the residual/LOO table.
6. **Kernel window**: kernel formula (6) rendered, fitted θ , and the LML grid with the three terms of (8).
7. **Next experiment**: ordered list of q set-points with predicted outcomes and $\Pr(\mathcal{F})$; export as a one-line protocol.
8. **Convergence**: HV_n , $\widehat{HV}_n$, $\exp(\log \alpha^*)/HV_n$ against n , with the thresholds; “converged — export” prompt.
9. **Simulation tab**: runs the full loop on `simulate.py` (FakeFrED) with the same config, and the qLogNEHVI vs. qNParEGO vs. Sobol comparison over several seeds. Offline only; never mixed with hardware data.

7.4 Extension to three objectives

Everything above is written over lists: d variables, m outcomes, M objectives. Going to 3D requires: (i) adding v_f to the variable list and switching `space.py` to the rotated coordinates $(\log R, \log v_f, T)$, whose rectangular bounds in raw space become a parallelogram expressed as linear inequality constraints passed to `optimize_acqf` via `inequality_constraints`; (ii) adding throughput $\dot{\ell} = \pi D_{\text{spool}} \omega_s$ (spooled length rate, recorded on every 2D run already) as a modeled outcome and maximized objective; (iii) a third reference-point coordinate. Hypervolume in $M = 3$ remains exact in BoTorch (box decomposition); cost grows but stays affordable at $n \leq 40$. The 2D runs transfer because throughput and v_f were logged.

8 BoTorch mapping (verified against the live documentation, 16 Sep 2026)

BoTorch v0.18.x. Signatures below were read from the current source/docs, not recalled. Claude Code must re-verify against the installed version before use and pin versions in `environment.yml`.

```
from botorch.models.gp_regression import SingleTaskGP # (train_X, train_Y, train_Yvar=
    None, likelihood=None,
from botorch.models.model_list_gp_regression import ModelListGP # covar_module=None,
    mean_module=None,
from botorch.models.transforms.outcome import Standardize # outcome_transform=DEFAULT(
    Standardize), input_transform=None)
from gpytorch.kernels import ScaleKernel, MaternKernel, RBFKernel # kernel choice per
    config
from gpytorch.mlls.sum_marginal_log_likelihood import SumMarginalLogLikelihood
from botorch import fit_gpytorch_mll
from botorch.acquisition.multi_objective.logei import
    qLogNoisyExpectedHypervolumeImprovement
# (model, ref_point, X_baseline, sampler=None, objective=None, constraints=None,
    X_pending=None,
# eta=1e-3, prune_baseline=False, alpha=0.0, cache_pending=True, max_iep=0,
# incremental_nehvi=True, cache_root=True, tau_relu=..., tau_max=1e-3, fat=True,
    marginalize_dim=None)
from botorch.acquisition.multi_objective.objective import GenericMCMultiOutputObjective #
    callable(samples, X) -> objectives
from botorch.sampling.normal import SobolQMCNormalSampler # SobolQMCNormalSampler(
    sample_shape=torch.Size([S]))
from botorch.optim.optimize import optimize_acqf # (acq_function, bounds, q, num_restarts
    , raw_samples,
                                # options=...,
                                inequality_constraints=[(
                                    idx, coef, rhs)],
                                # sequential=True) ; constraint
                                    form sum_i coef_i*x_idx_i
                                    >= rhs
from botorch.utils.sampling import draw_sobol_samples # (bounds, n, q)
from botorch.utils.transforms import normalize, unnormalize
from botorch.utils.multi_objective.pareto import is_non_dominated
from botorch.utils.multi_objective.hypervolume import Hypervolume # Hypervolume(ref_point
    ).compute(pareto_Y), maximization convention
```

Sign convention. Objectives are negated inside the `GenericMCMultiOutputObjective` and the reference point is passed negated and normalized; `Hypervolume` is evaluated on negated normalized fronts. Constraint callables return (11) on the raw (un-negated) outcome samples; negative means feasible. `prune_baseline=True`. Default $S = 128$ MC samples, `num_restarts=10`, `raw_samples=512`, `options={"batch_limit": 5, "maxiter": 200}`, double precision.

9 Implementation brief for Claude Code

This section is written to be handed to Claude Code verbatim, together with this PDF, the LaTeX source, the test CSV and the design record. It is deliberately explicit: the agent should be able to work for many sessions without asking what to do next.

9.1 Context

You are building **FrED-MOBO**, a local desktop application (Python, Anaconda, BoTorch) that runs a constrained two-objective Bayesian-optimization campaign on a fiber-extrusion testbed. The

users are a mechanical-engineering student and his advisor; the results go into a paper and later feed an LLM-based supervisory controller. The device produces one CSV per experiment; a separate power logger may produce a second file. The operator uploads these, declares whether the run failed, and the app proposes the next experiment. A campaign is 6 initial + about 20 adaptive runs spread over several days. The mathematics, decisions and module boundaries are fixed by this document; do not redesign them. Where the document is silent, choose the simplest option, record it in `docs/DECISIONS.md`, and continue.

9.2 Non-negotiables

1. qLogNEHVI from BoTorch ≥ 0.18 ; objectives negated internally (BoTorch maximizes); reference point fixed and normalized; `prune_baseline=True`; $S = 128$ Sobol QMC samples; double precision.
2. Independent GPs (`ModelListGP`) with an explicit `ScaleKernel(MaternalKernel(nu=2.5, ard))` or `ScaleKernel(RBFKernel(ard))` per config; outcome transform `Standardize`; noise inferred; refit from scratch after every accepted run; no warm start; no pickled models.
3. Objective 1 is $|\bar{d} - d^*|$ computed inside a `GenericMCMultiOutputObjective` on samples of $\bar{d}$; the GP models $\bar{d}$.
4. Constraints as BoTorch outcome-constraint callables (negative = feasible), two thresholds on one output, evaluated inside the MC expectation. Never multiply by a probability of feasibility.
5. Failures are declared by the operator; the software suggests, never decides (onset time, box shrinking, run rejection).
6. Failed runs never enter the objective GPs; they enter the constraint GP only in mode A.
7. Everything numeric is a config default editable at campaign setup; $n_0 = 6$ is the exception; kernel locks after the first adaptive run.
8. Target sessions share one dataset and one set of GPs; a new d^* changes only the objective transform and the diameter band, never the data or the hyperparameters; every export carries the map grids.
9. Save/resume: the campaign folder (SQLite + config JSON + copied raw files) is the only state; reopening restores everything and refits deterministically from data and seed.
10. Never write a BoTorch or GPyTorch signature from memory; read the installed docstring first and pin versions in `environment.yml`.
11. Do not build the UI until the backend runs end-to-end from the CLI on the emulator and all tests pass.

9.3 Repository layout (separate repo fred-mobo)

```
fred-mobo/
  CLAUDE.md # 40 lines max: stack, non-negotiables, pointer to docs/ARCHITECTURE.pdf
  environment.yml # pinned: python, torch, botorch, gpytorch, pyside6, matplotlib, pandas,
    scipy, pytest
  docs/ARCHITECTURE.pdf # this document docs/PLAN.md docs/PROGRESS.md docs/DECISIONS.md
  fred_mobo/
    config.py space.py extraction.py power.py models.py constraints.py
    acquisition.py pareto.py storage.py campaign.py simulate.py export.py
    ui/ (main_window.py, tabs/: settings, intake, pareto, design_space, gp, kernel, next,
      convergence, simulation)
  scripts/run_campaign_cli.py scripts/make_example_campaign.py
  tests/ (test_extraction.py, test_power.py, test_space.py, test_models.py,
    test_constraints.py,
      test_acquisition.py, test_pareto.py, test_storage.py, test_campaign_loop.py,
      test_export.py)
  data/examples/fred_experiment_test.csv
```


9.4 Milestones, in dependency order, each with its acceptance test

1. **M0 Scaffold.** Repo, environment, `CLAUDE.md`, the three `docs/` files, empty modules with docstrings, CI running `pytest`. Accept: `pytest` passes on an empty suite; `PLAN.md` lists M1–M9 with checkboxes.
2. **M1 config + space.** Typed `CampaignConfig` (variables as a list; objectives as a list with sign; constraint mode and thresholds; kernel; n_0 , budget, stopping; reference point; normalization ranges; D_{spool} , κ_f , $v_{f,0}$, d^* ; power column mapping). JSON round-trip. `space.py`: raw $\leftrightarrow$ unit cube; R from set-points; 3D rotated coordinates behind a flag with linear constraints. Accept: round-trip test; R test against hand calculation; 3D parallelogram constraint test.
3. **M2 extraction + power.** Algorithm 1; distinct-reading logic; equations (1)–(3); onset suggestion (Appendix A); power log alignment by offset; PM and PX sources. Accept: on `fred_experiment_test.csv` with declared BREAKAGE and suggested onset: onset 147.5 s, window $[0, 142.5]$ s, $\bar{d} = 0.427$ mm, $N = 193$, $\hat{\rho}_1 \approx 0.90$, $N_{\text{eff}} \approx 10$, trend warning true; PX heater power equals mean duty $\times P_{h,\text{max}}/100$; a synthetic power log with known offset is aligned to within 0.1 s.
4. **M3 simulate.** FakeFrED: $\bar{d} = d_{\text{in}}/\sqrt{R} \cdot g(T)$ with a mild temperature effect, power = $a + bT + c\omega_s^2$, breakage above $R_{\text{hi}}(T)$ and overflow below $R_{\text{lo}}(T)$, Gaussian run-to-run noise; writes device CSVs in the real schema (and optional power logs). Accept: generated files pass M2 extraction; failure regions match their definitions.
5. **M4 models.** Algorithm 2; kernel choice; `FitReport` with hyperparameters, LML, the LML grid over (ℓ_1, ℓ_2) , LOO residuals. Accept: on 30 emulator points the fitted length-scales are within a factor 3 of the emulator’s, LML grid maximum coincides with the fitted point.
6. **M5 constraints + acquisition + pareto.** Constraint callables from config (modes A/B); $\text{Pr}(\mathcal{F})$ grid; objective transform; qLogNEHVI construction and `optimize_acqf`; R -band suggestion logic; observed and posterior-mean fronts; hypervolume; convergence signals (15)–(17). Accept: on the emulator with 6+20 runs, hypervolume is monotone non-decreasing and the final observed front beats Sobol-only with the same budget over 5 seeds; no proposed candidate has $\text{Pr}(\mathcal{F}) < 0.05$ once the constraint GP has ≥ 8 points.
7. **M6 storage + campaign + export.** SQLite schema of Section 6; `CampaignController` implementing Algorithm 3; save/resume; JSON + Markdown + CSV export. Accept: close and reopen mid-campaign yields identical proposal and hypervolume; export validates against its JSON schema.
8. **M6b Target sessions.** Session table; predicted iso-diameter curve and front for a new d^* (Section 5.5); start-session action; per-session convergence; map grids and iso-diameter entries in the export. Accept: on the emulator, session 2 at a different d^* converges in fewer runs than session 1 under the same rule, and the front predicted before session 2 lies within its stated $\pm 2\sigma$ band of the front measured after it.
9. **M7 CLI end-to-end.** `run_campaign_cli.py` drives a full emulator campaign (intake, declare, fit, propose, converge, second target session, export) with no UI. Accept: runs unattended in < 5 min on CPU; produces the export files.
10. **M8 UI.** PySide6 windows of Section 7.3, English only, calling only `CampaignController`; Matplotlib QtAgg canvases; the $q > 1$ confirmation dialog; the kernel-lock behaviour. Accept: manual script in `docs/UI_CHECKLIST.md` completed; no computation in UI code.
11. **M9 Simulation tab + polish.** qLogNEHVI vs. qNParEGO vs. Sobol on the emulator over seeds inside the app; packaging instructions; final pass over `docs/`.

9.5 Pending hardware: what is built now and what will change

The current sensors are mounted on a PCB that is still being made. Every experiment that needs a current reading therefore waits, and the following parts of the software are **built now against the emulator only** and must be marked **hardware-unvalidated** in `DECISIONS.md` until the unit exists:

- Mode PM power ingestion (`power.py`): power-log parsing, column mapping, offset alignment, per-channel $V \cdot I$. The real header names, sample rate and marker format are *expected to differ* from the config defaults; keep them all in the config mapping, never hard-code them, and write the parser so that a schema change is a config edit plus one test-fixture update.
- Mode A constraint (spooler-current margin GP, thresholds τ_{lo}, τ_{hi} , failed-run margins): the logic and tests run on **FakeFrED**'s synthetic current channel; the thresholds and the monotonicity assumption (design record A1) are unverified until experiment M3b.
- The PCB request list (Section 2.4) may be only partly granted; the alignment fallbacks (timestamps, manual slide) must therefore be fully working, not stubs.
- The extruder constant $P_{f,0}$ and the load dependence of heater power (design record open item 6) are unknown until the unit exists; mode PX values are placeholders entered by the operator and flagged as such in the export.

Consequences for the plan: M2 and M5 include the PM/mode-A paths and their emulator tests; a later milestone **M10 Hardware bring-up** (re-open when the unit arrives) re-runs the M2 fixture on a real power log, adjusts the mapping, validates the alignment, and switches the campaign config from PX/B to PM/A only if the operator confirms. Until then the first campaign runs in PX/B, and the paper reports that.

9.6 Session protocol (mandatory, because credits may end a session at any time)

Start: read `docs/PROGRESS.md`, run `pytest`, restate the next step in one line. Work in small commits; after each unit of work update `PROGRESS.md` with what is done, what is verified by which test, and exactly where to resume (file, function, failing test). Record any deviation from this document in `DECISIONS.md` with the reason. End: commit with a message naming the next step. Never leave a half-written function without a `TODO(resume)` comment and a matching line in `PROGRESS.md`. Prefer finishing one milestone well over starting the next.

9.7 Conventions

Type hints everywhere; dataclasses for records; no global state; all randomness through a seeded `torch.Generator` / `numpy` generator passed explicitly; units in variable names (`d_mm`, `power_w`, `temp_c`); every public function has a docstring citing the equation or algorithm number of this document it implements; tests are the acceptance criteria above, not smoke tests.

9.8 What not to do

No web server, no Streamlit, no cloud calls, no Ax (raw BoTorch only, so every step is inspectable); no automatic failure detection that overrides the operator; no imputed values for failed runs; no recomputed reference point; no ARD-free kernels; no UI before M7; no signatures from memory.

A Optional onset-suggestion detector (assist only)

Kept from v1.0 because it was verified on the test file, but demoted to a suggestion the operator may accept or ignore. (i) $t_{\text{cross}} \leftarrow$ first t at which $d < d_{\text{break}} (= 0.5 d^*)$ or $d > d_{\text{over}} (= 2 d^*)$ persists for ≥ 3 s. (ii) Plateau median $\tilde{d}$ and MAD on $[t_{\text{start}}, t_{\text{cross}} - 30 \text{ s}]$; rolling 5-reading median $\tilde{d}_k$; suggested onset $\leftarrow$ last $t < t_{\text{cross}}$ with $|\tilde{d}_k - \tilde{d}| \leq 4 \text{ MAD}$. On the test file: $t_{\text{cross}} = 167.2$ s, suggested onset 147.5 s, matching the visual collapse; a bare threshold would end the window at 162 s and swallow the thinning ramp ($\hat{\rho}_1 = 0.98$, $N_{\text{eff}} = 2$). This is why, when the operator's declared time differs from the suggestion, the intake plot shows both markers.

B Sources

1. Rasmussen, C. E. & Williams, C. K. I. (2006). *Gaussian Processes for Machine Learning*. MIT Press. Used: predictive equations (2.23)–(2.24), log marginal likelihood (2.30), Matérn-5/2 (4.17), ARD (§5.1), ML-II gradient (5.8)–(5.9). [Equation numbers verified against the Project PDF in earlier sessions; the copy in this Project could not be re-opened in this session.]
2. Frazier, P. I. (2018). A Tutorial on Bayesian Optimization. arXiv:1807.02811. Used: BO loop, EI (§4.1, with the known erratum in its eq. (8)), noisy evaluations, parallel evaluations (eq. 14) and constraints (§5).
3. Daulton, S., Balandat, M., Bakshy, E. (2020). Differentiable Expected Hypervolume Improvement for Parallel Multi-Objective Bayesian Optimization. NeurIPS 33. arXiv:2006.05078. Used: qEHVI, box decomposition, reparameterized MC, sequential greedy.
4. Daulton, S., Balandat, M., Bakshy, E. (2021). Parallel Bayesian Optimization of Multiple Noisy Objectives with Expected Hypervolume Improvement. NeurIPS 34. arXiv:2105.08195. Used: qNEHVI (13), constraint handling inside the expectation, pruning of the baseline.
5. Ament, S., Daulton, S., Eriksson, D., Balandat, M., Bakshy, E. (2023). Unexpected Improvements to Expected Improvement for Bayesian Optimization. NeurIPS 36. arXiv:2310.20708. Used: log reformulation (14).
6. Hvarfner, C., Hellsten, E. O., Nardi, L. (2024). Vanilla Bayesian Optimization Performs Great in High Dimensions. ICML. arXiv:2402.02229. Used: current BoTorch default priors (kernel decision, Section 3).
7. Emmerich, M., Giannakoglou, K., Naujoks, B. (2006). Single- and multiobjective evolutionary optimization assisted by Gaussian random field metamodels. IEEE Trans. Evol. Comput. 10(4), 421–439. Used: expected hypervolume improvement.
8. Zitzler, E. & Thiele, L. (1999). Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach. IEEE Trans. Evol. Comput. 3(4), 257–271. Used: hypervolume indicator (12).
9. Gelbart, M. A., Snoek, J., Adams, R. P. (2014). Bayesian Optimization with Unknown Constraints. UAI. arXiv:1403.5607. Used: outcome-constraint formulation.
10. Knowles, J. (2006). ParEGO: a hybrid algorithm with on-line landscape approximation for expensive multiobjective optimization problems. IEEE Trans. Evol. Comput. 10(1), 50–66. Used: baseline only.
11. Jones, D. R., Schonlau, M., Welch, W. J. (1998). Efficient Global Optimization of Expensive Black-Box Functions. J. Global Optim. 13, 455–492. Used: EI closed form (primary source, given the Frazier erratum).
12. Box, G. E. P., Jenkins, G. M., Reinsel, G. C. *Time Series Analysis: Forecasting and Control*. Used: AR(1) effective-sample-size correction (3).
13. BoTorch documentation, v0.18.1: https://botorch.org/docs/multi_objective, <https://botorch.org/docs/constraints>, constrained MOBO tutorial, and API reference <https://botorch.readthedocs.io>. Accessed 16 Sep 2026. Used: Section 8.
14. Grupo Estudiantil FrED (2026). *FrED MOBO — experimental design record* (Project file). Used: constraint architecture, instrumentation, firmware findings, open items.